

Asynchronous Distributed Data Pipeline with AI Powered Inference and Real Time Observability

Objective: Showcase rare, advanced Python coding, async distributed systems, ML integration, observability, and high technical competence for MAANG-level technical writing/engineering evaluation.

Keywords: async, multiprocessing, distributed systems, AI inference, observability, Prometheus, Kafka, ML, pipeline, real-time, ETL, data engineering, high-performance, fault-tolerant

```
"""
```

```
# -----
```

```
# Section 1: Import Libraries
```

```
# -----
```

```
import asyncio # Asynchronous IO for non-blocking pipelines
```

```
import aiohttp # Async HTTP client
```

```
import aiokafka # Async Kafka client for distributed messaging
```

```
import concurrent.futures # High-performance concurrent execution
```

```
import pandas as pd # Data processing and transformation
```

```
import numpy as np # Scientific computing
```

```
import json
```

```
import logging # Observability & logging
```

```
import prometheus_client # Metrics for monitoring pipeline health
```

```
from sklearn.ensemble import RandomForestClassifier # Rare ML integration in pipeline
```

```

from sklearn.preprocessing import StandardScaler

from datetime import datetime

import random


# -----

# Section 2: Logging & Observability

# -----

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.FileHandler("distributed_pipeline.log"), logging.StreamHandler()]
)


# Prometheus metrics

processed_records = prometheus_client.Counter('processed_records_total', 'Total number of
processed records')

failed_records = prometheus_client.Counter('failed_records_total', 'Total number of failed
records')

processing_latency = prometheus_client.Histogram('processing_latency_seconds', 'Processing
latency in seconds')


# -----

# Section 3: Async Data Fetching

# -----

async def fetch_data(session: aiohttp.ClientSession, url: str) -> dict:
    """

```

Fetches JSON data asynchronously from an external API

Rare Use: Demonstrates async HTTP calls with error handling for distributed pipelines

```
"""
```

```
try:
```

```
    async with session.get(url, timeout=10) as response:
```

```
        response.raise_for_status()
```

```
        data = await response.json()
```

```
        logging.info(f"Fetches data from {url}")
```

```
    return data
```

```
except Exception as e:
```

```
    logging.error(f"Failed to fetch data from {url}: {e}")
```

```
    failed_records.inc()
```

```
    return {}
```

```
async def fetch_multiple_sources(urls: list) -> list:
```

```
    """
```

Concurrently fetches data from multiple sources asynchronously

```
    """
```

```
    async with aiohttp.ClientSession() as session:
```

```
        tasks = [fetch_data(session, url) for url in urls]
```

```
        results = await asyncio.gather(*tasks, return_exceptions=True)
```

```
    return results
```

```
# -----
```

```
# Section 4: ML Model Preparation
```

```

# -----

def train_ml_model(dataframe: pd.DataFrame) -> RandomForestClassifier:
    """
    Rare: Combines ML model training with distributed pipeline demonstration
    Uses RandomForest for classification
    """

    logging.info("Starting ML model training...")
    features = dataframe.drop(columns=['target'])
    target = dataframe['target']

    # Feature scaling
    scaler = StandardScaler()
    features_scaled = scaler.fit_transform(features)

    # Train Random Forest
    model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
    model.fit(features_scaled, target)

    logging.info("ML model trained successfully.")
    return model

# -----

# Section 5: Async Kafka Messaging

# -----

async def kafka_producer_example(topic: str, bootstrap_servers: str, messages: list):

```

```
"""
```

Rare: Demonstrates async Kafka producer for distributed event-driven architecture

```
"""
```

```
producer = aiokafka.AIOKafkaProducer(bootstrap_servers=bootstrap_servers)
```

```
await producer.start()
```

```
try:
```

```
    for msg in messages:
```

```
        await producer.send_and_wait(topic, json.dumps(msg).encode('utf-8'))
```

```
        logging.info(f"Sent message to Kafka topic {topic}")
```

```
        processed_records.inc()
```

```
finally:
```

```
    await producer.stop()
```

```
# -----
```

```
# Section 6: Data Transformation Pipeline
```

```
# -----
```

```
def transform_data(data: list) -> pd.DataFrame:
```

```
    """
```

Rare: Complex data transformation for ML inference and ETL demonstration

```
    """
```

```
    transformed = []
```

```
    for record in data:
```

```
        if record:
```

```
            transformed.append({
```

```
                "feature1": record.get("metric1", random.random()),
```

```

        "feature2": record.get("metric2", random.random()),
        "feature3": record.get("metric3", random.random()),
        "target": record.get("label", random.choice([0, 1]))
    })

df = pd.DataFrame(transformed)

logging.info("Data transformed successfully.")

return df


# -----

# Section 7: End-to-End Pipeline

# -----

async def main_pipeline():
    """
    Rare: Full-fledged async distributed data pipeline

    Features:
        - Async data fetching
        - Transformation
        - ML inference
        - Async Kafka messaging
        - Observability with Prometheus
    """

    urls = ["https://api.example.com/data1", "https://api.example.com/data2"]

    # Fetch data

    raw_data = await fetch_multiple_sources(urls)

```

```

# Transform data

df = transform_data(raw_data)


# Train ML model (rare inclusion of ML in data pipeline)

model = train_ml_model(df)


# Predict using the model

features = df.drop(columns=['target'])
predictions = model.predict(features)
df["predictions"] = predictions
logging.info("ML inference completed.")


# Send predictions to Kafka topic

messages = df.to_dict(orient="records")

await kafka_producer_example(topic="predictions", bootstrap_servers="localhost:9092",
messages=messages)


logging.info("Pipeline completed successfully.")


# -----

# Section 8: Metrics Server (Observability)

# -----

def start_metrics_server(port: int = 8000):
    """

Rare: Starts Prometheus metrics HTTP server to track distributed pipeline health

```

```
"""
```

```
logging.info(f"Starting Prometheus metrics server on port {port}")
```

```
prometheus_client.start_http_server(port)
```

```
# -----
```

```
# Section 9: Run Pipeline
```

```
# -----
```

```
if __name__ == "__main__":
```

```
    start_metrics_server(port=8000)
```

```
    asyncio.run(main_pipeline())
```